

Self-Stabilizing Master–Slave Token Circulation and Efficient Topology Computation in a Tree of Arbitrary Size*

Wayne Goddard and Pradip K Srimani
School of Computing
Clemson University
Clemson, SC 29634-0974
{goddard, srimani}@cs.clemson.edu

Abstract

Self-stabilizing algorithms represent an extension of distributed algorithms in which nodes of the network have neither coordination, synchronization, nor initialization. We consider the model introduced by Lee et al. where there is one designated master node and all other nodes are anonymous and as simple as possible. We provide here a master–slave algorithm that achieves token circulation in a tree, and using that, enables the master node to compute the topology of the tree. We show that the complexity of the algorithm is at most $2n$ rounds ($O(n^2)$ steps), and the space complexity at the slave nodes is almost constant in the sense that storage needed collectively at all n slave nodes is $2n$ bits.

1 Introduction

Self-stabilizing algorithms represent an extension of distributed algorithms in which nodes of the network have neither coordination, synchronization, nor initialization. Thus self-stabilization handles an unlimited number of transient faults. Each node participates in the distributed algorithm based on local knowledge: its own state and the states of its immediate neighbors. The objective is to achieve some global objective—a predicate defined on the states of all the nodes in the network—based on local actions where individual nodes have no global knowledge about the network.

Recently, Lee, Tzeng, and Huang [1] offered a new model for self-stabilization. They considered the prob-

lem of determining the number n of nodes in a ring using constant space. Well, actually this is impossible, since even writing down the value n requires $O(\log n)$ space. So, to obtain a “space-efficient” algorithm, they designated one node that would determine the answer, but required all other nodes to be as simple as possible.

We call their algorithm a *master–slave* algorithm. The network has a unique master node, and all other nodes are anonymous; that is, no node can explicitly use a node ID to make any decision or to take any action; we use node identifiers for reference purpose only. The algorithms are semi-uniform, in that the same program is run at all slave nodes.

In this paper, we provide a master–slave algorithm for the problem of token circulation in a tree. The token circulation protocol uses three states on each link and the randomized version stabilizes in expected $O(n \log n)$ steps. Once the system stabilizes with one token, we then provide a polling algorithm to compute the topology of the tree at the master node. This (deterministic) algorithm stabilizes in $O(n^2)$ steps.

2 Terminology and Model

In a self-stabilizing algorithm, every node has a set of local variables whose contents specify the local state of the node. The state of the entire system, called the *global state*, is the union of the local states of all the nodes in the system. Each node has only a partial view of the global state (and this view depends on the connectivity of the system). Furthermore, there is no synchronization, not even a common starting point. Yet, the system arrives at a desirable global final state (legitimate state).

*This work has been supported by NSF grant # CCF-0832582

A self-stabilizing algorithm is specified as a collection of (production) rules for each node. Each rule has a trigger *condition* and an *action*. The trigger condition is a boolean predicate on the state of the node and the state of its neighbors; the action or *move* specifies a change in the state of the node's variables. A node is said to be *privileged* at a particular time if the trigger condition on one or more of its rules is satisfied.

In order to analyze the correctness of the algorithm and its time complexity under a worst-case scenario, a self-stabilizing algorithm is assumed to face an adversarial daemon. The daemon plays the role of both scheduler and adversary. In the literature, there are several daemons. We focus here on a *central daemon*, also known as a serial daemon. A central daemon picks a single arbitrary privileged node to move (execute the rule's action) at each step. In contrast, the *distributed daemon* taps a nonempty subset of the privileged nodes to move at each step. We show that the *algorithms work under both daemons*.

We assume that we have an arbitrary tree network whose edges are bidirectional; an arbitrary node of the tree is designated as the master node labeled r and the other nodes labeled 1 through $n - 1$ in an arbitrary fashion. These labels are used only for reference purposes; the proposed algorithms do not make use of the node identifiers. The master node is unique, it knows it's the master and the other nodes are anonymous (we also call them slave nodes). The master node r does not have any predecessor while any slave node i has a single predecessor and multiple (including zero) children; the nodes are aware of only its adjacent nodes and none is aware of which of its adjacent nodes is its predecessor. As is customary in self-stabilization [2], we work in the *shared-variable* or *state-reading* model in which a node can directly read its neighbors' variables. In our design of the data structure at each node, a neighbor can read all the neighbors' variables but use those informations selectively according to the protocol's requirements.

The algorithms for token circulation and for tree topology computation do not terminate. Rather, they are guaranteed to eventually reach and remain in a state where there is a single token in circulation, or the master node knows the topology of the tree, respectively. This is sometimes referred to as the algorithm being *dynamic*.

3 Master-Slave Token Circulation in Trees

We assume we have a tree network with the designated root node of the tree labeled as the master node, node r , and the other nodes in order labeled 1 through $n - 1$. These labels are used only for reference purposes; the master node is unique, it knows it's the master and the other nodes are anonymous.

Each node i has δ_i many neighbors, exactly one of which is its predecessor/parent and the remainder are its children in the tree (the root/master node does not have any predecessor). Note that we do not assume that a node knows which node is its parent. For each node i , there is some fixed but arbitrary ordering of the neighbors of node i numbered $0, 1, \dots, \delta_i - 1$; the ordering of neighbors of a node is independent of such ordering of neighbors at any other node, i.e., the ordering of neighbors at any node is in no way related to the ordering of the neighbors any other node in the tree. We define $\Delta = \max_{i \in T} \delta_i$ as the maximum degree of a node in the tree T .

We generalize the master-slave token circulation algorithm in a unidirectional ring provided in [3] for adoption in a bidirectional tree network. Note that the algorithm in [3] was in turn was based on an earlier algorithm of Afek and Brown [4, 5] (which was actually proposed for a different model in a different context; authors assumed some initialization of the state of the nodes, and FIFO queues on the links of the network) We call the result the AB algorithm.

In the AB algorithm, each node has δ_i (stored) token variables $t_i^0, t_i^1, \dots, t_i^{\delta_i-1}$ each of which can be in one of three states, say $\mathbb{A}$, $\mathbb{B}$, or $\mathbb{C}$; we also use notations s_j , $j = 0, 1, \dots, \delta_i - 1$ to denote the partial states of the neighbors of node i as sensed (read) at node i . We observe

- Consider an undirected edge (i, j) in the three between nodes i and j – we view this edge as two parallel directed edges: $i \rightarrow j$ and $j \rightarrow i$. Assume node j is the k -th neighbor of node i , $1 \leq k \leq \delta_i - 1$ (according to the unique numbering of neighbors at node i) and node i is the k' -th neighbor of node j , $1 \leq k' \leq \delta_j - 1$ (according to the unique numbering of neighbors at node j); then, s_i^k is the value of the stored variable $t_j^{k'}$ (at node j) as read by node i from its neighbor j and similarly $s_j^{k'}$ is the value of the stored variable t_i^k (at node i) as read by node j from its neighbor i . In other words, node

i and its neighbor node j communicates with each other by using the registers t_i^k (stored at node i) and $t_j^{k'}$ (stored at node j); the s -variables are used only for notational convenience.

- Pairs of corresponding token variables at neighboring nodes essentially simulate directed edges in the tree and essentially imposes a directed Eulerian cycle on the tree graph starting at the root node using a depth first walk along the tree starting and ending at root node r .

The rule for any slave node i is as follows: For any j , $j = 0, 1, \dots, \delta_i - 1$, if $t_i^j \neq s_i^k$, then $t_i^j = s_i^k$, where $k = j - 1 \pmod{\delta_i}$. Node i maintains a cyclic ordering of its neighbors; when it senses that the token variable at some of its neighbors is different from its own token reserved for its next neighbor in that cyclic order, we say that node i has received a token from its neighbor and it changes its own token for the next neighbor to match with the sensed token; we say node i passes the received token to its next neighbor.

The rule for the master node is somewhat different: Assume the root node r has δ_r neighbors, numbered in a cyclic order $1, 2, \dots, \delta_r - 1$; like any slave node, we say the root node receives a token when it senses that the token variable at some of its neighbors is different from its own token reserved for its next neighbor in that cyclic order; it becomes privileged when it receives a token from any of its neighbors. When privileged by receiving a token from any of its neighbors $0, 1, \dots, \delta_r - 2$, it behaves like a slave node, i.e., passes the token to its next neighbor. The interesting question is the rule for how the master node compares its token t_i^0 with the sensed value $s_r^{\delta_r-1}$ – the master node is privileged (receives the token) when $(t_i^0 = s_i^{\delta_i-1})$ and it changes its token t_i^0 differently – we call this the **Update rule**, and defer discussion about this until we have established some general properties of the algorithm. The resulting AB algorithm for a tree T is given in Figure 1.

The desired goal behavior is that the master node changes its t^0 -value, this value circulates the entire directed Eulerian cycle, and then the master node changes its value again. Indeed, we have a legal final configuration if and only if the number of privileged nodes is 1.

Definition 1 *Any global system state is **legal** or **stable** if and only if there is exactly one token in the unidirectional Eulerian cycle, i.e., the number of privileged nodes (the master or a slave) is exactly one.*

We note immediately that the algorithm remains live, and that over time the number of privileged nodes can only decrease:

Lemma 1 (a) *The AB algorithm cannot terminate.*
(b) *The number of privileged nodes cannot increase.*

Proof: (a) If every slave node is not privileged, then this means that $t_0 = t_1 = t_2 = \dots = t_{n-1}$. Thus the master node is privileged.

(b) It is immediate that the number of privileged nodes can never go up—in any unidirectional cycle algorithm, as a node moves, it becomes unprivileged and only its successor can be made privileged. $\square$

Thus we only need argue that eventually the number of privileged nodes decreases. To get stuck in an infinite loop, what enters the master node must continually be the same as what leaves it. Thus, as Afek and Brown [4] show, for convergence to a legal state, it is sufficient to choose a sequence for the t_0 such that the sequence of tokens is aperiodic. We provide a simpler proof for the necessity of aperiodicity of the sequence of tokens at the master node for convergence of the global state to a legal one starting from an arbitrary state.

Lemma 2 *If the the sequence of distinct values of t_0 is periodic, there always exists some initial global system state in a tree such that the AB algorithm does not converge to a legal final configuration.*

Proof: The proof is based on a similar theorem given in [3], by replacing an unidirectional ring with the unidirectional Eulerian cycle imposed on the tree in this case. We omit the details for lack of space. $\square$

3.1 The Update Rule

We now turn to discussion of the **Update** rule. This amounts to discussion of ways of generation of aperiodic sequences. One complex idea is to use a *square-free word* [6] (such as that given by Thue [7]), which has the property that no subword is repeated, and in particular, is not periodic.

But a simple idea is that the master node maintains two counters: r and s (for round and step). The **Update** rule is given in Figure 2.

```

Rule for the Master Node ( $i = r$ ):
    {
        if ( $t_i^0 = s_i^{\delta_i-1}$ )
            then change  $t_i^0$  according to formula Update;
        for ( $j = 1; j < \delta_i; j++$ )
            if ( $t_i^j \neq s_i^{j-1 \bmod \delta_i}$ ) then
                 $t_i^j = s_i^{j-1 \bmod \delta_i}$ ;
    }

Rule for a Slave Node ( $i > 0$ ):
    {
        for ( $j = 0; j < \delta_i; j++$ )
            if ( $t_i^j \neq s_i^{j-1 \bmod \delta_i}$ ) then
                 $t_i^j = s_i^{j-1 \bmod \delta_i}$ ;
    }

```

Figure 1. AB Algorithm for Token Circulation in a Tree

If we were allowed to initialize the master node (to $t = \mathbb{A}$ and $s = r = 0$), then this rule would generate

$\mathbb{A}, \mathbb{B}, \mathbb{C}, \mathbb{A}, \mathbb{B}, \mathbb{A}, \mathbb{B}, \mathbb{C}, \mathbb{A}, \mathbb{B}, \mathbb{A}, \mathbb{B}, \mathbb{A}, \mathbb{B}, \mathbb{C}, \mathbb{A}, \dots$

This sequence is clearly aperiodic, as noted in [5].

However, this algorithm may require unbounded memory at the master node. Moreover, because the master node is not initialized, it can start in a state where $s = 0$ and r is huge. It will then produce alternating $\mathbb{A}, \mathbb{B}$ for a very long time, and the daemon can avoid stabilization with such a sequence. Thus there is no upper bound on the stabilization time of the AB algorithm for the above deterministic update.

This situation can be avoided if one knows a bound B on the size of the cycle (or possibly even if this algorithm is used in the midst of an algorithm that computes the size B of the cycle). For, in this case, one can wrap r around to zero when it reaches $B + 1$ (a period longer than B is irrelevant). Thus one uses $O(\log B)$ bits at the master node. We note that this is probably best possible.

However, one simple practical way to implement the AB algorithm is to add randomness, as observed in [5]. Specifically, allow the master node to choose the next t_0 uniformly at random from the two other values. It follows that if there are multiple tokens in circulation, what the master node receives from node $n - 1$ is not what the master just sent. Thus there is a 50-50 chance that the next token will match, and thus disappear.

As in [1], we define a round to be the time taken by a token to circulate around the unidirectional cycle; that is, a round is equivalent to n steps.

Deterministic Update:

```

if (  $t = \mathbb{A}$  )
     $t = \mathbb{B}$ ;
else if (  $t = \mathbb{A}$  and  $s < r$  )
     $t = \mathbb{B}; s++$ ;
else if (  $t = \mathbb{B}$  and  $s = r$  )
     $t = \mathbb{C}$ ;
else
     $t = \mathbb{A}; s = 0; r++$ 

```

Figure 2. Deterministic Update

Randomized Update:

Let $t \in \{\mathbb{A}, \mathbb{B}, \mathbb{C}\}$ — t uniformly at random.

Figure 3. Randomized Update

Theorem 1 *Under Randomized Update, the expected time to convergence of one circulating token is $O(n \log n)$ steps, regardless of the behavior of the daemon.*

Proof: We give just a sketch of the proof. The goal of the daemon is to maintain multiple tokens in circulation. Most moves simply advance a token from one node to the next without changing the set of tokens. The moves that can potentially destroy a token are (a) tapping the master node; or (b) advancing one token to overwrite the next (for example, if three consecutive slave nodes have $\mathbb{A}, \mathbb{B}, \mathbb{C}$ and the daemon taps the middle node then the $\mathbb{B}$ -token dies).

Consider a situation where the daemon performs (a). Say the first token in circulation is $\mathbb{A}$ and the daemon and its predecessor are in $\mathbb{B}$. Then there is a 50% chance that the token generated by the daemon is the same as that, and hence the token dies. Thus, if the daemon never does (b), then the expected number of tokens generated by the master node before we get to one token is at most $2n$. Thus, approximately, if we start with n tokens, after all have passed through the master we should expect to have about $n/2$ tokens, and so on, and so after $O(\log n)$ rounds we would have only one token.

Now, to provide a full proof, we need to make the previous sentence more mathematically accurate. But more

importantly, we need to deal with the fact that the daemon can use (b) to destroy specific tokens. For example, this behavior is especially noticeable if the sequence of tokens is the deterministic one given above. There if the daemon destroys every $\mathbb{C}$ that is generated and one associated $\mathbb{B}$, it can force $O(n)$ rounds to convergence (even if we are allowed to initialize the master node). So one needs to show that option (b) does not change things significantly. We omit the details. $\square$

Remark 1 *The AB algorithm handles a distributed daemon too. Indeed, if the distributed daemon does not choose all the nodes simultaneously, then in any unidirectional cycle algorithm, this is equivalent to a sequence of individual moves (move the frontmost node first). So the only issue with considering a distributed daemon is what happens if the daemon selects all nodes simultaneously. This can, of course, only happen if every node is privileged. And we have just argued that that state cannot continue indefinitely.*

4 Tree Topology

Using the token circulation algorithm above, we can develop a master-slave algorithm to calculate the topology of the tree (as a rooted tree) at the master node. The key idea is that one can encode any rooted tree by recording the ups and downs as one traverses the $2(n-1)$ links of the Eulerian cycle. The algorithm uses the token to accumulate this information, which the master node then uses to calculate the topology in a space efficient way.

4.1 Trees and Sequences

As before, we assume a tree with a master (root) node r . We want the master node to compute the topology of the tree. We need to have a convenient representation of the topology that can be gathered at the root node in a space-efficient way.

Recall the directed Eulerian cycle on the tree starting and ending at the root node using a depth first walk. Assume we denote a link of the cycle by $\mathbb{D}$ (down) if the link is from parent to child, and denote the link by $\mathbb{U}$ (up) if the edge is from a child to parent. Thus, the Eulerian cycle can be represented by a unique sequence $\mathcal{S}$ of length $2(n-2)$ from symbols $\{\mathbb{D}, \mathbb{U}\}$.

Note that one can readily recover the tree topology from this sequence $\mathcal{S}$. Maintain a partial rooted tree

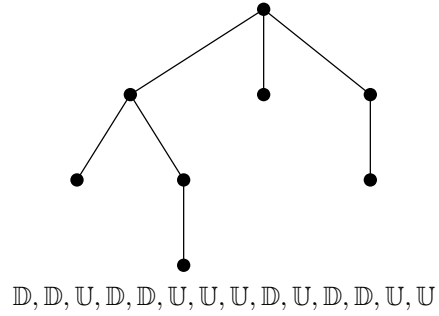


Figure 4. A rooted tree and its sequence

and a current node v . Each time a $\mathbb{D}$ is encountered, add a child to v and move v to that child; every time a $\mathbb{U}$ is encountered, move v to its parent. Figure 4 gives an example of a tree and the associated sequence (assuming the children of each node are ordered from left to right).

Also, this sequence can be denoted as a sequence of 0's and 1's and the sequence can be computed and stored at the master node as an integer. Our approach to compute the topology of the tree at the root node is to compute the topology one bit at a time. The master node (root r of the tree) computes the topology one bit per round (in a right-to-left fashion); thus, if n is the size of the tree, $O(n)$ circulation of a single token would suffice to gather the complete information about the Eulerian cycle of length $2(n-1)$.

Note that the directed Eulerian cycle is different for different choices of orderings at the nodes, thus giving a different ordered tree, but the unordered tree remains the same. The above connection is actually well-known. In particular, if we think of $\mathbb{D}$ as left parenthesis and $\mathbb{U}$ as right parenthesis, then the sequence $\mathcal{S}$ we obtain is one of balanced parentheses. And, it is well known that both these combinatorial objects are counted by the Catalan numbers. This is discussed for example in [8] or sequence A000108 in [9].

Lemma 3 *There is a bijection between the set of rooted ordered trees with n nodes and the set of all sequences of length $2(n-2)$ of balanced parentheses.*

(Note that the root actually only wants the unordered tree; there are exponentially many of these (On the order of 2.9558^n ; see sequence A000081 in [9]), so that determining these one bit at a time, like we do below for the ordered tree, will still take linear rounds.)

4.2 Accumulating the Sequence

Our approach to compute the topology of the tree at the root node is to compute the sequence $\mathcal{S}$ one bit at a time. The master node (root r) computes one bit per round; thus, $O(n)$ circulations of a single-bit token suffice to gather the complete information about the Eulerian cycle. (Also, this sequence $\mathcal{S}$ can be denoted as a sequence of 0's and 1's and the sequence can be computed and stored at the master node as an integer.)

To describe our distributed algorithm conceptually, we ignore self-stabilization for the time being. The concept of the distributed algorithm is as follows. Every node has a Loaded bit for each link coming into it. The master node starts by sending out a Reset token that clears all Loaded bits. After that the master node sends out a Data token with a single bit that gathers the next bit in the sequence.

The first round this Data token gathers the information of the first link in the directed Eulerian cycle. In general, when a (slave) node receives an (empty) Data token along a link whose corresponding Loaded bit is false, it sets the Loaded bit and loads the edge information: if the token comes from its parent, it loads a $\mathbb{D}$, otherwise a $\mathbb{U}$. And the key point is that a slave node does not have to know who its parent is: rather, the first time it sees the token, the token comes from the parent. So when the slave receives the empty Data token and all its Loaded bits are false, it loads a $\mathbb{D}$, otherwise it loads a $\mathbb{U}$.

If a (slave) node receives a token along a link whose corresponding Loaded bit is true, it simply passes on the token unchanged. And so on. Thus, in each round, when the master node receives the token back, it gathers the information about exactly one link of the Eulerian cycle. In $2(n-1)$ rounds, the master node gathers the complete information of the directed Eulerian cycle of links.

So that the master knows that the process is complete, one adds another “Pristine” state to the token; if the master node gets the Pristine state back, then it knows all the links are accounted for and the algorithm is complete.

4.3 Protocol TOPOLOGY

We first run the AB algorithm to reach a legal configuration. Of course, nodes do not know when the AB

algorithm is finished; so we just combine the two. In the TOPOLOGY algorithm, each node i maintains following data structure:

- δ_i token variables $t_i^0, t_i^1, \dots, t_i^{\delta_i-1}$, each of which can be in one of three states, $\mathbb{A}$, $\mathbb{B}$, or $\mathbb{C}$
- A 4-state status variable $status_i$ that can have any of four values *Pristine*, *Reset*, *Down*, or *Up*
- Each node i has a boolean flag $loaded_i^j$ corresponding to its j -th neighbor $0 \leq j \leq \delta_i - 1$. Thus, each slave node i has δ_i boolean bits of storage.
- The unique master node ($i = 0$) has two integer variables *count* and *pos* to store the topology of the tree.

Token circulation is done using the previous self-stabilizing AB algorithm, using the t -tokens. We view the status variables as if the node has a status token e.g., a privileged node receives a status token from its unique predecessor in the directed Eulerian cycle, adjusts its own *status* variable according to the rules, thereby sending the status token to the next node in the directed Eulerian cycle. When the previous AB algorithm stabilizes, a single t -token circulates along the directed Eulerian cycle. In a complete round in the cycle, a node becomes privileged multiple times, each time with a different unique predecessor and a different unique successor along the cycle. The complete details for the TOPOLOGY protocol are shown in Figure 5.

Note: When the master node r is privileged under the condition $t_r^0 = s_r^{\delta_r-1}$ (i.e. the t -token has come back to the master node after one traversal of the Eulerian cycle and it needs to generate a new t -token), it sends out a (1) $\mathbb{R}$ status token iff it receives a $\mathbb{P}$ status token from its predecessor node in the Eulerian cycle; (2) otherwise a privileged master node always sends out $\mathbb{P}$ status token to its successor node in the directed cycle. When the master node is privileged under the condition $t_r^j \neq s_r^{j-1}$, for some j , $1 \leq j \leq \delta_r - 1$ (i.e. the t -token has come back to the master node before completing one traversal of the Eulerian cycle), it relays the received status token (from its predecessor) unchanged to its successor.

4.4 Analysis

A configuration is *legal* if there is exactly one t -token in the system (either $\mathbb{A}$, $\mathbb{B}$, or $\mathbb{C}$ type), and so exactly one

Rule for the Master Node (r):

if ($t_r^0 = s_r^{\delta_r-1}$) then

- Set $t^0 = \text{choose by Update from } \{\mathbb{A}, \mathbb{B}, \mathbb{C}\} - \{s_r^{\delta_r-1}\}$;
- [Assume k denotes the final neighbor of node i]
- if $status_k = \mathbb{R}$ then
 - { $status_r = \mathbb{P}$; $count = 0$; $pos = 0$; }
- else if $status_k = \mathbb{P}$ then
 - { $status_r = \mathbb{R}$; }
- else if $status_k = \mathbb{U}$ then
 - { $status_r = \mathbb{P}$; $pos++$; }
- else if $status_k = \mathbb{D}$ then
 - { $status_r = \mathbb{P}$; $count++ = 2^{pos}$; $pos++$; }

else

- if ($t_r^j \neq s_r^{j-1}$, for some j , $1 \leq j \leq \delta_r - 1$) then
 - { $t_r^j = s_r^{j-1}$;
 - $status_r = status_k$; } [k denotes the j -th neighbor of the root]

Rule for a Slave Node ($i > 0$):

if ($t_i^j \neq s_i^{j-1 \bmod \delta_i}$, for some j , $0 \leq j \leq \delta_i - 1$) then

- Set $t_i^j = s_i^{j-1 \bmod \delta_i}$; [say, k denotes the j -th neighbor of node i]
- if $status_k = \mathbb{R}$ then
 - { $status_i = \mathbb{R}$; $loaded_i^j = \text{false}$ for all j with $0 \leq j < \delta_i$; }
- else if $status_k = \mathbb{P} \wedge \text{not } loaded_i^j$ for all j then
 - { $status_i = \mathbb{D}$; $loaded_i^j = \text{true}$; }
- else if $status_k = \mathbb{P} \wedge \text{not } loaded_i^j$ then
 - { $status_i = \mathbb{U}$; $loaded_i^j = \text{true}$; }
- else $status_i = status_k$;

Figure 5. Master-Slave Algorithm TOPOLOGY to compute the topology of an arbitrary Tree

node (either slave or master) is privileged. When the privileged node moves, the system transitions to a legal configuration where the next node along the underlying directed Eulerian cycle is privileged. By Theorem 1, the global system state will reach a *legal configuration* in expected $O(n \log n)$ steps starting from an arbitrary system state.

Lemma 4 *When the master node r is privileged under the condition $t_r^0 = s_r^{\delta_r-1}$ and sends out an $\mathbb{R}$ status token, each slave node in the tree resets all its loaded bits to false in the next round whereupon the master node becomes privileged again with $t_r^0 = s_r^{\delta_r-1}$.*

Proof: When a privileged slave node i receives a $\mathbb{R}$ status token, it sets its $loaded_i^j$ bits, corresponding to its j -th neighbor $0 \leq j \leq \delta_i - 1$ to be false and sends an $\mathbb{R}$ status token to the next node in the directed Eu-

lerian cycle (by the first clause in the rules for slaves). The process continues all the way to the master node at the end of the Eulerian cycle. $\square$

Lemma 5 *In a round, if the master does not send an $\mathbb{R}$ status token, then (1) the number of false loaded bits at slave nodes cannot increase; and (2) moreover, exactly one of the false loaded bits of the alive slave nodes at the beginning of the round will be true by the end of the round.*

Proof: A slave node i , when privileged with its j -th loaded bit true, does not change that flag and relays the status token received from its predecessor in the Eulerian cycle to its successor. Thus (1) is true. A slave node i , when privileged with its j -th loaded bit false, (a) resets that bit to true after sending out a non- $\mathbb{P}$ status token on receiving a $\mathbb{P}$ status token; (b) does

not change its $loaded_i^j$ bit after relaying the received status token on receiving a non- $\mathbb{P}$ status token. Thus, exactly one false *loaded* bit can be true in one round. $\square$

Lemma 6 *Starting with any good configuration, all false loaded bits at the slave nodes will be true by at most $O(n)$ rounds or $O(n^2)$ steps.*

Proof: In any configuration, the number of false loaded bits at the slave nodes is at most $2(n-1)$ (there are $n-1$ slave nodes in the given tree). Thus this lemma follows from Lemma 5. $\square$

Lemma 7 *When all loaded bits at each slave node are true, the master node will receive a $\mathbb{P}$ status token in at most one round.*

Proof: The master node becomes privileged with $t_r^0 = s_r^{\delta_r-1}$ once each round. If it does not receive a $\mathbb{P}$ status token, then it will relay a $\mathbb{P}$ status token. This status token will circulate around the directed Eulerian cycle since slave nodes receiving the token along a link with the *loaded* bit true will just pass the status token and so will reach the master node after completing the cycle. $\square$

Putting all this together, we get the following:

Theorem 2 *Starting from an arbitrary legal configuration, the variable count at the master node will contain the sequence of the tree in at most $2n$ rounds ($O(n^2)$ steps).*

Proof: By Lemmas 6 and 7, the system will reach a special legal configuration where all *loaded* bits at each slave node are dead and the master node is privileged with an $\mathbb{R}$ status token in at most $2n$ rounds.

At this point, the master node sends out a $\mathbb{R}$ status token. This, in one round, clears each *loaded* flag and every slave node has its $status_i$ set to $\mathbb{R}$. On receiving the $\mathbb{R}$ status token, the privileged master node, in the next round, sends out a $\mathbb{P}$ status token. One link will become visited. At the end of the round, the master node will receive a status token $\mathbb{D}$ or $\mathbb{U}$ depending on the data. The master adjusts the counter variable, and starts a new round in a similar way. After $2n-2$ rounds, the master node gets back the status token $\mathbb{P}$; it now knows that the counter contains the correct sequence, and so knows the topology. $\square$

5 Conclusion

We have proposed a self-stabilizing master-slave that achieves token circulation in a tree, and enables the master node to compute the topology of the tree. The algorithm is space efficient in that the space used collectively at all nodes is $2(n-1)$ binary bits where n is the number of nodes including the master node.

References

- [1] K.-C. Lee, C.-H. Tzeng, and S.-T. Huang. A space-efficient self-stabilizing algorithm for measuring the size of ring networks. *Inform. Process. Lett.*, 94:125–130, 2005.
- [2] N. Guellati and H. Kheddouci. A survey on self-stabilizing algorithms for independence, domination, coloring, and matching in graphs. *Journal of Parallel Distributed Computing*, 70:406–415, 2010.
- [3] W. Goddard and P.K. Srimani. Self-stabilizing master-slave token circulation and efficient size-computation in a unidirectional ring of arbitrary size. In *Parallel Distributed Processing, Workshops and Phd Forum (IPDPSW), 2010 IEEE International Symposium on*, pages 1–8, april 2010.
- [4] Y. Afek and G.M. Brown. Self-stabilization of the alternating-bit protocol. In *Proceedings of the 8th symposium on Reliable Distributed Systems (The Sixteenth Conference of Electrical and Electronics Engineers in Israel)*, pages 80–83, 1989.
- [5] Y. Afek and G.M. Brown. Self-stabilization over unreliable communication media. *Distributed Computing*, 7:27–34, 1993.
- [6] Squarefree word. http://en.wikipedia.org/wiki/Squarefree_word.
- [7] A. Thue. Über die gegenseitige lage gleicher teile gewisser zeichenreihen. *Norske vid. Selsk. Skr. Mat. Nat. Kl.*, 1:1–67, 1912.
- [8] Richard P. Stanley. *Enumerative combinatorics. Vol. 2*, volume 62. Cambridge University Press, Cambridge, 1999.
- [9] Neil Sloane. The on-line encyclopedia of integer sequences. url-<http://www.research.att.com/njas/sequences/>.